Sprint 0 --- Technical Proposal

FareShare - campus ridesharing for IBA students

Team: ***Uroos Fatima*** *(31094),* ***Muhammad Shayan Hussain*** *(30077),* ***Waniya Jabeen*** *(31624) | Software Engineering, Fall 2026*

Note: Our initial Trello Board had issues with list creation hence we had to make a new one.
here's the link for the New Trello Board:
https://trello.com/invite/b/6abf27da019e2084f8e19aa1/ATTI0b8b56e0a6e76fcba25cd5ea7b40c969932E463E/fareshare-fall26

1. Project Proposals

Interfaces

- Software: Flutter mobile app communicating with Firebase (Auth + Firestore) through the Firebase SDK; handles account creation/login and create/read/update/delete operations on Trips, Trip Requests and Vehicles
- Hardware: no custom hardware; push notifications (request accepted/declined, trip confirmed) via Firebase Cloud Messaging; area selection is manual (dropdown/text) rather than GPS-based
- UX: Signup/Login → Home (browse/search trips) → Post Trip → Trip Details/Request → My Requests → Profile

Scope

FareShare is a student carpool coordination app for the IBA community. Students can create verified accounts, post and manage trips, search available trips, request seats, and coordinate trip confirmation. The approved scope includes three CRUD entities (Trips, Trip Requests, and Vehicles), with User data supporting account/profile functionality, gender filtering, and a completed-trips counter. Real payment processing, live GPS tracking, and real-time in-app chat are outside the core Sprint 1–2 scope.

Cloud Usage

- Firebase used as a fully managed Backend-as-a-Service; no self-hosted server
- Firebase Auth handles signup/login; the @khi.iba.edu.pk restriction is checked in the app and enforced again by a Cloud Function so it cannot be bypassed
- Cloud Firestore stores all entities and their relationships as collections/documents
- Firebase Cloud Messaging sends push notifications for trip request status changes

Server Details

- No custom backend server; the Flutter app communicates directly with Firebase services through the Firebase SDK.
- Firestore security rules control document access, while lightweight Cloud Functions handle logic that should not be trusted to the client, such as the email-domain safeguard, counters, and notification triggers.

Tech Stack

Layer	Choice
Frontend	Flutter (Dart)
Backend	Firebase Auth + Cloud Functions
Database	Cloud Firestore
Notifications	Firebase Cloud Messaging
Version Control	GitHub (main + feature branches, reviewed pull requests)
Backlog / Sprint Board	Trello

2. Constraints & Non-Functional Requirements

Category	Requirement
Performance - Search	Trip search results shall load within 2 seconds under normal campus Wi-Fi or mobile data.
Performance - Posting	Posting a new trip shall complete within 3 seconds, with a loading indicator shown until the it is confirmed.
Security - Signup	Only accounts with an @khi.iba.edu.pk email shall be created; any other domain shall be rejected both in the app and by a server-side Cloud Function.
Security - Passwords	The system shall never store or handle passwords directly; all authentication shall go through Firebase Auth.
Platform / Device Support	Android 10 (API 29) or later and iOS 15 or later.
Availability / Reliability	With no internet or a slow connection, the app shall show a loading state or a clear error message and shall not freeze.
Technical Constraint - Backend	The backend shall remain within Firebase's free tier (Spark plan), with no custom server infrastructure.
Technical Constraint - Timeline	Delivery shall meet Sprint 1 (Oct 31) and Sprint 2 (Nov 28) with 3 team members balancing other coursework; scope is kept deliberately tight.

3. Use Case Diagram

Covers the two primary actors, Trip Manager and Companion (one student account can act as both, depending on the trip), plus the System as a secondary actor performing automated email-domain verification.

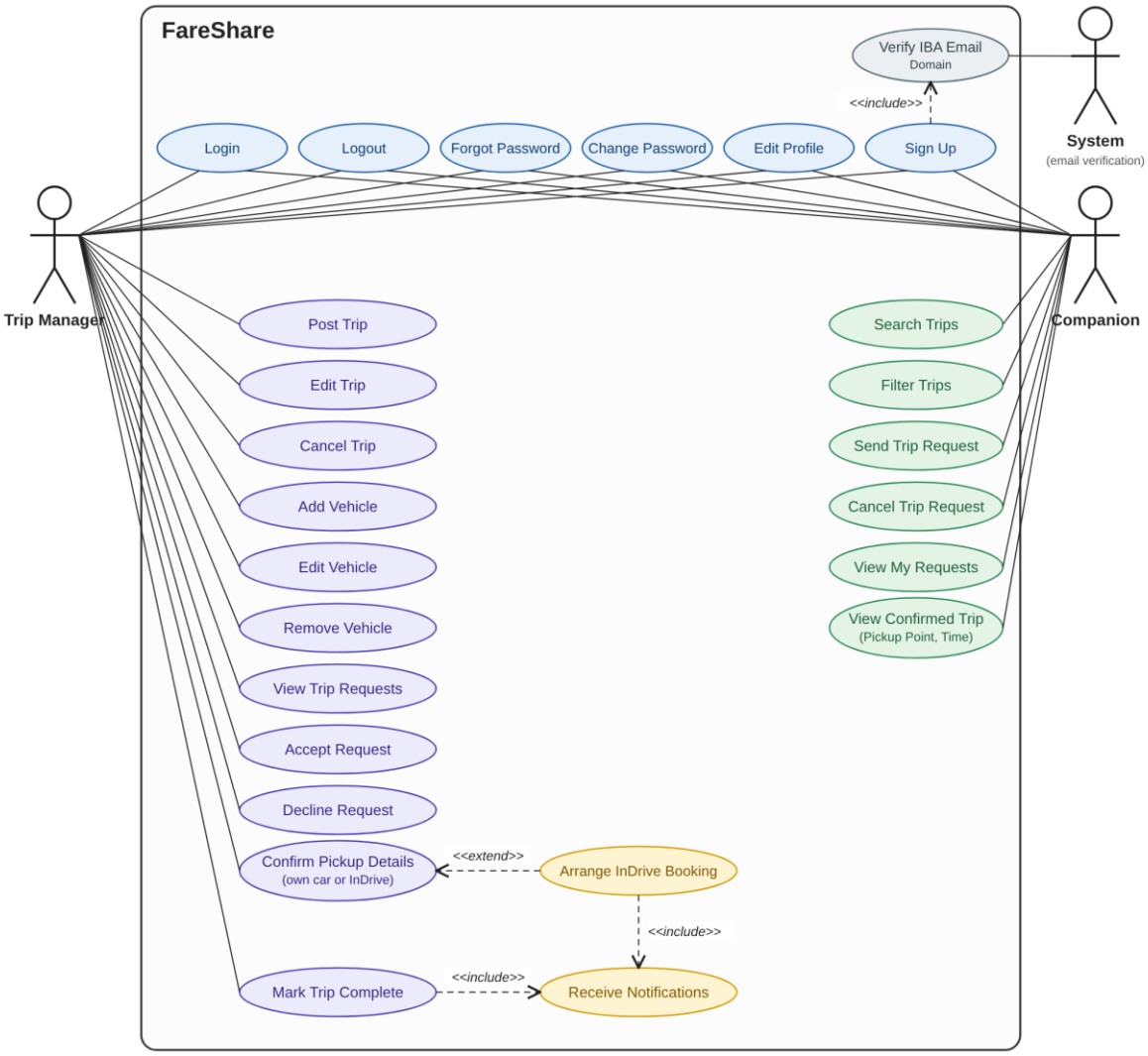


Figure 1. FareShare use case diagram

4. Entity Relationship Diagram (ERD)

Covers the 3 required CRUD entities (Trip, Trip Request, Vehicle) plus the User entity, which holds profile data, the gender filter and the completed-trips counter. User and Trip are related many-to-many through Trip Request, the junction entity.

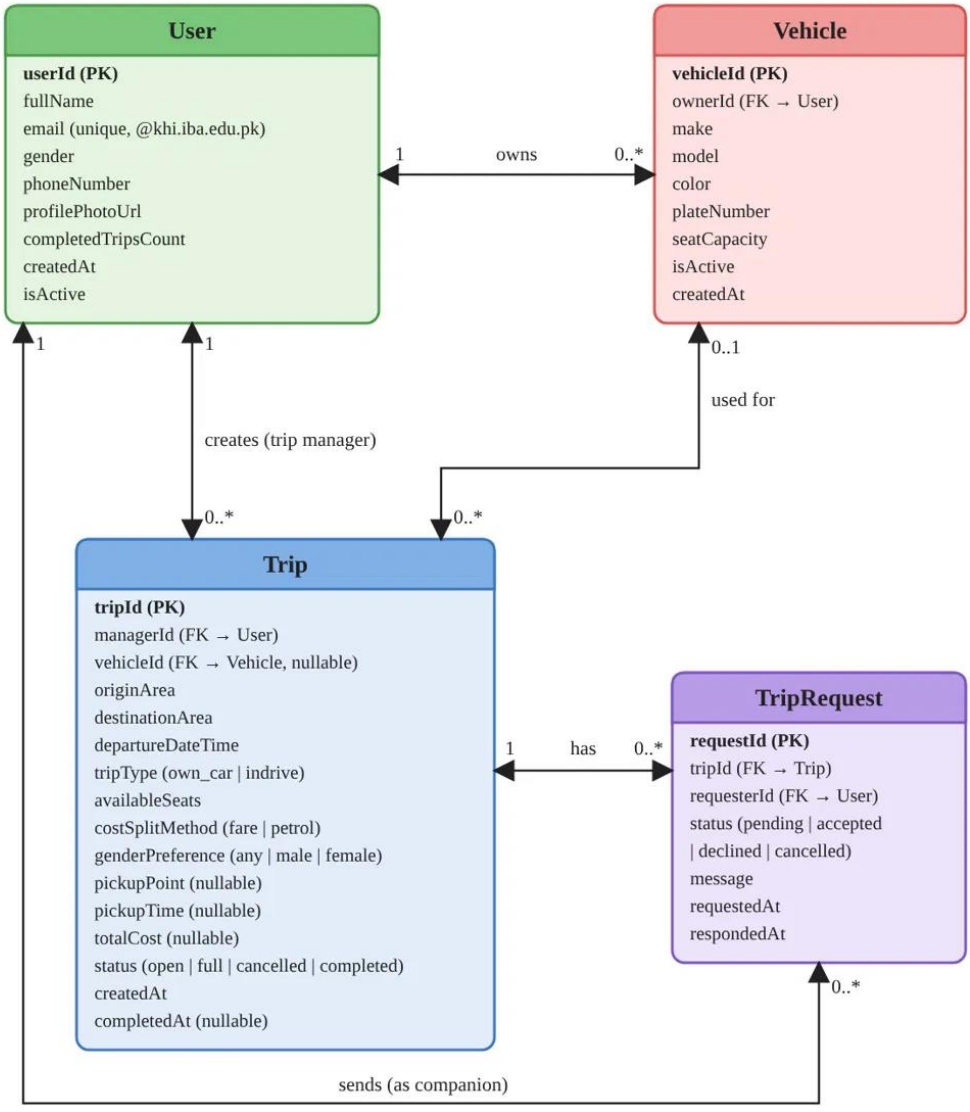


Figure 2. FareShare entity relationship diagram

5. Architecture - Just Enough

The minimum needed to start building safely. Expected to evolve across sprints. The Flutter app talks directly to Firebase Auth, Cloud Firestore and Cloud Messaging; Cloud Functions provide the server-side logic that must not be trusted to the client.

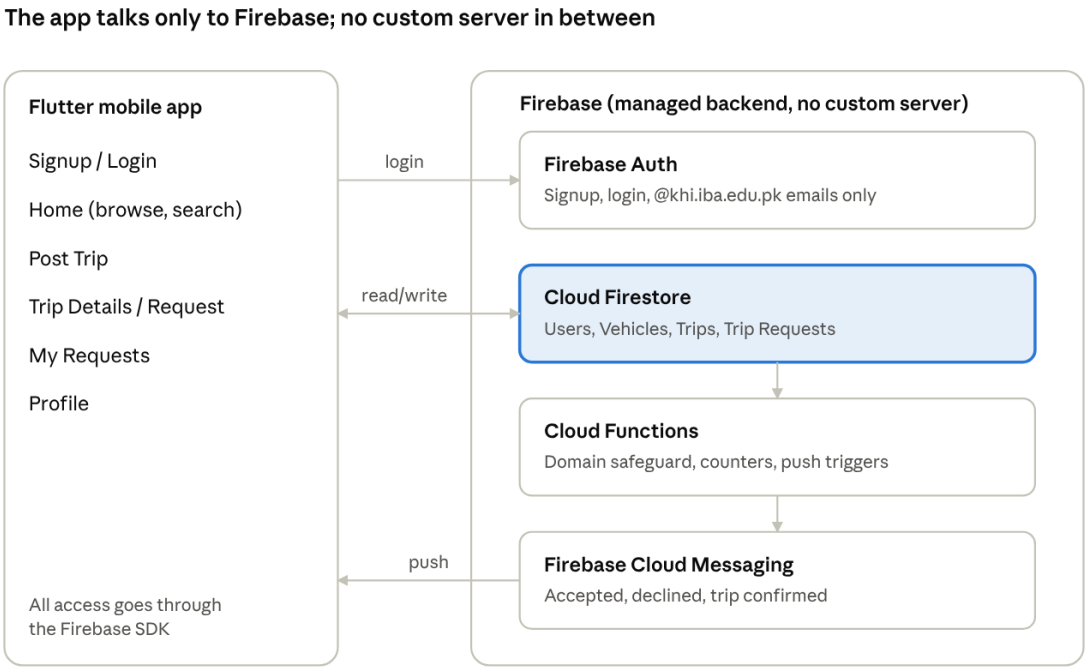


Figure 3. FareShare high-level architecture

Git Branching Strategy

- main -- always stable; merges only through reviewed pull requests
- feature/<story-id>-<short-name> -- one branch per user story, merged into main via pull request